

# Formatos de dados e bancos de dados

Eduardo Adame

**Ciência de Dados**

24 de abril de 2026



# Uma breve história dos dados



## Antes dos computadores

- Registros em papel, fichas perfuradas
- Décadas de 1950–60: fitas magnéticas, acesso sequencial
- 1970: Edgar Codd propõe o **modelo relacional**
- 1979: primeiro SQL comercial (Oracle)

## Era moderna

- Anos 2000: explosão da web ⇒ **NoSQL**
- 2006: Hadoop / HDFS — processamento distribuído
- 2010s: Data Warehouses na nuvem (Redshift, BigQuery)
- 2020s: formatos colunares open-source (**Parquet**, Arrow)

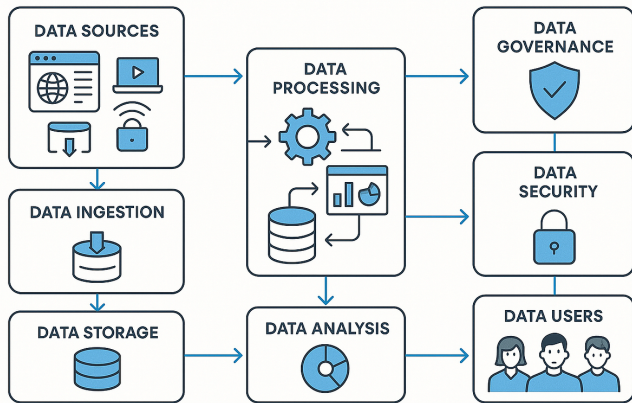
### Lei de Codd

Dados devem ser acessados pelo *conteúdo*, não pela localização física.

# O ecossistema atual de armazenamento



## DATA ECOSYSTEM



# CSV — o mínimo denominador comum



## Vantagens

- Legível por humanos e qualquer ferramenta
- Universal — Excel, R, Python, SQL, ...
- Fácil de inspecionar e depurar

## Desvantagens

- Sem tipos — tudo vira string
- Sem esquema embutido
- Lento para arquivos grandes
- Delimitador pode colidir com os dados

## Armadilhas clássicas

- Encoding: UTF-8 vs. latin-1
- Separador decimal: 3.14 vs. 3,14
- Datas: DD/MM/AAAA vs. YYYY-MM-DD
- NULL, NA, ‘”, - — qual é o faltante?



## Listing 1: Estrutura típica

```
1 {  
2   "aluno": "Ana Silva",  
3   "turma": "A",  
4   "notas": [8.5, 7.0, 9.2],  
5   "endereco": {  
6     "cidade": "Rio de Janeiro",  
7     "uf": "RJ"  
8   }  
9 }
```

## Características

- **Hierárquico** — aninhamento nativo
- Tipos primitivos: string, número, bool, null
- Padrão da web (APIs REST)
- Sem esquema obrigatório

## Variantes

- NDJSON / JSON Lines — uma linha por registro
- GeoJSON — dados geoespaciais

# Parquet — o formato colunar



## Columnar Database

| Name  | Age | Country |
|-------|-----|---------|
| Alice | 30  | USA     |
| Bob   | 25  | UK      |
| Carol | 40  | Canada  |
|       |     |         |
|       |     |         |

Stored together on disk

## Row-oriented Database

| Name  | Age | Country |
|-------|-----|---------|
| Alice | 30  | USA     |
| Bob   | 25  | UK      |
| Carol | 40  | Canada  |
|       |     |         |
|       |     |         |

Stored together on disk



## Listing 2: Leitura e escrita com Pandas

```
1 import pandas as pd
2
3 # Escrita
4 df.to_parquet("dados.parquet", index=False)
5 df.to_parquet("dados.parquet", compression="snappy") # padrão
6
7 # Leitura
8 df = pd.read_parquet("dados.parquet")
9
10 # Leitura seletiva de colunas (vantagem colunar!)
11 df = pd.read_parquet("dados.parquet", columns=["nome", "media"])
```

- **Compressão nativa:** Snappy, Gzip, Zstd
- **Esquema embutido:** tipos preservados automaticamente
- **Predicate pushdown:** filtra antes de carregar na memória
- Base do ecossistema Apache Arrow / DuckDB / Spark



## CRUD

As quatro operações básicas:

- **C**reate — inserir registros
- **R**ead — consultar
- **U**ppdate — atualizar
- **D**eleate — remover

## Relações

- 1:1 — pessoa ↔ passaporte
- 1:N — turma ↔ alunos
- N:M — alunos ↔ disciplinas

## ETL vs. ELT

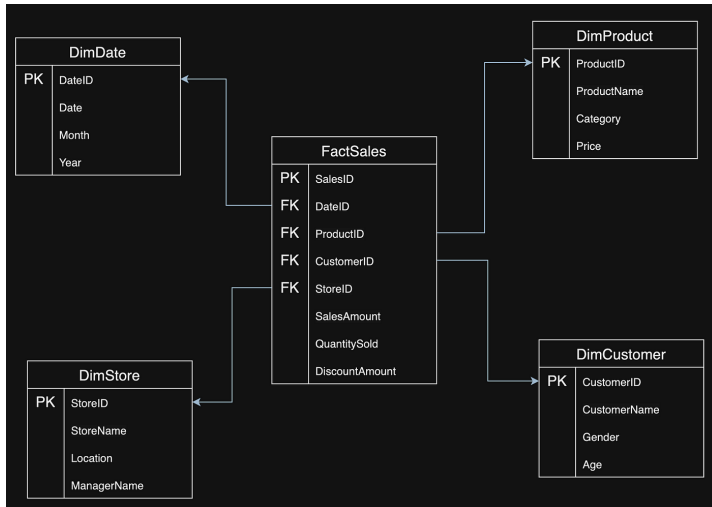
- **ETL**: Extrai → Transforma → Carrega  
Dado limpo antes de entrar no destino
- **ELT**: Extrai → Carrega → Transforma  
Dado bruto no Data Lake; transforma depois

### Tendência moderna

ELT favorecido por Data Lakes e ferramentas como dbt.



# Star Schema — modelagem analítica



# Data Lake, Data Warehouse e Data Swamp



## Data Lake

- Dados brutos, qualquer formato
- Barato, escalável
- Esquema definido *na leitura*
- Ex: S3, GCS, ADLS

## Data Warehouse

- Dados limpos e estruturados
- Esquema definido *na escrita*
- Otimizado para análise (OLAP)
- Ex: BigQuery, Redshift, Snowflake

## Data Swamp

- Data Lake **sem governança**
- Dados sem catálogo ou linhagem
- Ninguém sabe o que está lá
- Resultado de descuido operacional

## Lakehouse

Tendência atual: unificar Lake + Warehouse com formatos transacionais abertos (Delta Lake, Apache Iceberg).



## Listing 3: As quatro operações essenciais

```
1  -- CREATE (inserir)
2  INSERT INTO alunos (nome, turma) VALUES ('Ana', 'A');
3
4  -- READ (consultar)
5  SELECT nome, media FROM alunos WHERE media >= 6.0
6  ORDER BY media DESC LIMIT 10;
7
8  -- UPDATE (atualizar)
9  UPDATE alunos SET turma = 'B' WHERE id = 42;
10
11 -- DELETE (remover)
12 DELETE FROM alunos WHERE id = 42;
```

- SQL é **declarativo** — você diz *o quê*, não *como*
- Padronizado (SQL-92, SQL:2023), com dialetos por SGBD
- Habilidade transferível: SQLite, Postgres, MySQL, BigQuery, Spark SQL



## Listing 4: JOIN e GROUP BY

```
1  -- Left join: todos os alunos, com professor se existir
2  SELECT a.nome, a.media, p.professor
3  FROM alunos a
4  LEFT JOIN professores p ON a.turma = p.turma;
5
6  -- Agregação por turma
7  SELECT
8      turma,
9      COUNT(*)                AS total,
10     ROUND(AVG(media), 2)     AS media_turma,
11     SUM(CASE WHEN media >= 6 THEN 1
12          ELSE 0 END)         AS aprovados
13  FROM alunos
14  GROUP BY turma
15  ORDER BY media_turma DESC;
```



## Listing 5: Uso direto em Python

```
1 import sqlite3
2 conn = sqlite3.connect("escola.db")    # cria o arquivo
3 cur  = conn.cursor()
4
5 cur.execute("""
6     CREATE TABLE IF NOT EXISTS alunos (
7         id      INTEGER PRIMARY KEY AUTOINCREMENT,
8         nome    TEXT    NOT NULL,
9         turma   TEXT,
10        media   REAL
11    )
12 """)
13
14 cur.execute("INSERT INTO alunos (nome, turma, media) VALUES (?, ?, ?)",
15             ("Ana", "A", 8.5))
16 conn.commit()
17 rows = cur.execute("SELECT * FROM alunos").fetchall()
18 conn.close()
```

## SQLite — banco de dados em arquivo



- Banco **embutido** — sem servidor, sem instalação
- Arquivo único portátil (.db ou .sqlite)
- Ideal para protótipos, apps desktop, testes



## Listing 6: Subindo containers com Docker Compose

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     environment:
5       POSTGRES_USER: admin
6       POSTGRES_PASSWORD: secret
7       POSTGRES_DB: escola
8     ports: ["5432:5432"]
9
10  mysql:
11    image: mariadb:11
12    environment:
13      MARIADB_ROOT_PASSWORD: secret
14      MARIADB_DATABASE: escola
15    ports: ["3306:3306"]
```



## Listing 7: Iniciar e parar

```
1 docker compose up -d      # sobe em background
2 docker compose down       # para e remove containers
3 docker compose down -v    # remove volumes (dados) também
```





## Listing 8: PostgreSQL

```
1  # Dump completo
2  pg_dump -U admin escola > escola_backup.sql
3
4  # Dump comprimido (recomendado para produção)
5  pg_dump -U admin -Fc escola > escola_backup.dump
6
7  # Load
8  psql -U admin escola < escola_backup.sql
9  pg_restore -U admin -d escola escola_backup.dump
```



## Listing 9: SQLite

```
1 import sqlite3, pathlib
2
3 # Dump (SQL puro)
4 conn = sqlite3.connect("escola.db")
5 pathlib.Path("backup.sql").write_text(
6     "\n".join(conn.iterdump())
7 )
8
9 # Load
10 conn2 = sqlite3.connect("restaurado.db")
11 conn2.executescript(pathlib.Path("backup.sql").read_text())
```

# SQLAlchemy — ORM e Core



SQLAlchemy oferece duas camadas: **Core** (SQL expressivo em Python) e **ORM** (mapeamento objeto-relacional).

## Listing 10: Conexão e inserção com Core

```
1 from sqlalchemy import create_engine, text
2
3 # SQLite
4 engine = create_engine("sqlite:///escola.db")
5
6 # PostgreSQL (via Docker)
7 engine = create_engine(
8     "postgresql+psycopg2://admin:secret@localhost:5432/escola"
9 )
10
11 with engine.connect() as conn:
12     conn.execute(text(
13         "INSERT INTO alunos (nome, turma, media) VALUES (:n, :t, :m)"
14     ), {"n": "Bruno", "t": "B", "m": 7.2})
15     conn.commit()
```



## Listing 11: Definindo modelos

```
1 from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, Session
2
3 class Base(DeclarativeBase): pass
4
5 class Aluno(Base):
6     __tablename__ = "alunos"
7     id: Mapped[int] = mapped_column(primary_key=True)
8     nome: Mapped[str]
9     turma: Mapped[str]
10    media: Mapped[float]
11
12 Base.metadata.create_all(engine)
13
14 with Session(engine) as session:
15     session.add(Aluno(nome="Carla", turma="C", media=9.1))
16     session.commit()
17
18    alunos = session.query(Aluno).filter(Aluno.media >= 7).all()
```



## Listing 12: read\_sql e to\_sql

```
1 import pandas as pd
2 from sqlalchemy import create_engine
3
4 engine = create_engine("sqlite:///escola.db")
5
6 # Carregar tabela inteira
7 df = pd.read_sql("SELECT * FROM alunos", engine)
8
9 # Consulta com filtro
10 df = pd.read_sql(
11     "SELECT nome, media FROM alunos WHERE turma = 'A'",
12     engine
13 )
14
15 # Escrever DataFrame de volta ao banco
16 df_novo.to_sql("alunos_limpos", engine,
17               if_exists="replace", index=False)
```



- `if_exists`: “fail” (padrão), “replace”, “append”
- `chunksize` — escrita em lotes para tabelas grandes
- Para leituras grandes: `pd.read_sql(..., chunksize=10_000)`

## Listing 13: Polars lê Parquet e faz SQL nativo

```
1 import polars as pl
2 # Leitura lazy (não carrega tudo na memória)
3 df = pl.scan_parquet("dados.parquet")
4 # API expressiva - compilada para execução otimizada
5 resultado = (
6     df
7     .filter(pl.col("media") >= 6.0)
8     .group_by("turma")
9     .agg(
10         media_turma=pl.col("media").mean(),
11         total=pl.len(),
12     )
13     .sort("media_turma", descending=True)
14     .collect() # executa o plano
15 )
16 # SQL diretamente sobre DataFrames
17 ctx = pl.SQLContext(alunos=df)
18 ctx.execute("SELECT turma, AVG(media) FROM alunos GROUP BY turma")
```

# Além do single-node: processamento distribuído



## Quando escalar?

- Dados **não cabem na RAM**
- Transformações demoram horas
- Necessidade de execução contínua (streaming)

## Ferramentas modernas

- **DuckDB** — SQL analítico in-process, multi-thread
- **Polars** — paralelismo nativo em CPU
- **Dask** — Pandas paralelo com API familiar
- **PySpark** — Apache Spark via Python

## Regra de bolso

| Tamanho  | Ferramenta      |
|----------|-----------------|
| <1 GB    | Pandas / Polars |
| 1–100 GB | Polars / DuckDB |
| >100 GB  | Spark / Dask    |

### Avisos

Spark tem overhead de cluster. Para <50 GB num servidor, DuckDB supera Spark em velocidade e simplicidade.





## Listing 14: DuckDB lê Parquet sem carregar na RAM

```
1 import duckdb
2
3 # Consulta direta sobre arquivo - sem pd.read_parquet()!
4 resultado = duckdb.sql("""
5     SELECT
6         turma,
7         AVG(media)    AS media_turma,
8         COUNT(*)      AS total
9     FROM read_parquet('dados.parquet')
10    WHERE media IS NOT NULL
11    GROUP BY turma
12    ORDER BY media_turma DESC
13 """).df()    # converte para DataFrame Pandas
14
15 # Registrar DataFrame Pandas como tabela virtual
16 duckdb.register("alunos", df_pandas)
17 duckdb.sql("SELECT * FROM alunos WHERE media >= 7").show()
```



## Listing 15: Sessão local e operações básicas

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql import functions as F
3 spark = SparkSession.builder \
4     .appName("escola") \
5     .master("local[*]") \      # usa todos os cores locais
6     .getOrCreate()
7
8 # Leitura
9 df = spark.read.parquet("dados.parquet")
10 # Transformações (lazy - não executam até .show() ou .write)
11 resultado = (
12     df.filter(F.col("media") >= 6.0)
13     .groupBy("turma")
14     .agg(F.avg("media").alias("media_turma"),
15         F.count("*").alias("total"))
16     .orderBy("media_turma", ascending=False)
17 )
18 resultado.show()
19 spark.stop()
```

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!